

GROUPE

M1-LDFS

(Master 1 en « Développement & Innovation »)

Titre Professionnel**« Expert en Informatique et Système d'Information »
(Niveau 7)**ANNEE SCOLAIRE

« 2025-2026 »

Projet de fin d'année**Conception et développement d'une application de suivi de
la Coupe du monde de football FIFA 2026**Contenu :

Contexte et problématique
Benchmarking et analyse de la stratégie
Identification & Analyse des Besoins
Cahier des charges technico-fonctionnelles
Gestion de projet et planification/suivi des tâches
L'architecture du projet
Conception du système, de la BDD et de l'IHM
Développement de l'application
Sécurité de l'application et des données
DevOps
IA & Data

Proposé par

Ryadh HADJ MOKHNECHE



Attention !

- Le travail demandé est à réaliser par groupe de 4 personnes.
- La soutenance devant le jury est individuelle
- L'élève devra utiliser une présentation PPT individuelle.

Remarques !!

Prenez le temps de bien lire le sujet. Cela vous évitera d'atteindre un certain niveau de travail pour se rendre compte plus tard qu'il fallait peut-être faire les choses de manière différente.

Prenez le temps aussi de bien identifier et analyser les besoins avant de passer à la conception puis au développement.

I. Partie 1 : Description générale

I.1. Contexte du mini-projet :

Dans le cadre de la formation en **conception et développement d'applications informatiques**, ce mini-projet a pour objectif de placer les apprenants dans une situation proche d'un **projet réel**, avec des contraintes de temps, d'organisation et de qualité.

Les étudiants devront concevoir et développer une **application de suivi de la Coupe du monde de football 2026**, comparable aux solutions existantes proposées par des acteurs comme **Google**, des sites sportifs ou des applications mobiles spécialisées.

Le projet couvre l'ensemble de la compétition : phase de groupes, seizièmes de finale, huitièmes de finale, quarts de finale, demi-finales, petite finale, finale.

I.2. Objectifs :

Ce mini-projet vise à évaluer et renforcer les compétences suivantes :

- **Analyse du besoin** à partir d'un sujet fonctionnel ouvert
- **Conception d'une application** (structure, écrans, données, flux)
- **Travail en équipe** (répartition des rôles, coordination, communication)
- **Développement applicatif** (web ou mobile)
- **Gestion du temps et des priorités** dans un délai court
- **Mise à jour et affichage de données en temps réel**
- Respect des **bonnes pratiques de développement** (lisibilité, structuration, maintenabilité)

I.3. Description générale de l'application attendue

L'application développée devra permettre à un utilisateur de :

- Consulter les **matchs de la Coupe du monde 2026**,
- Visualiser les **résultats, scores et informations associées**,
- Suivre l'évolution de la compétition **en temps réel**.

Les données affichées devront être dynamiques et se mettre à jour automatiquement (les étudiants pourront s'inspirer de sites ou applications existants pour comprendre les usages et attentes utilisateurs).

Le choix du type d'application et des technologies à utiliser sont laissés libres :

- Application **web** ou **mobile** ou **hybride**, selon les compétences et choix techniques du groupe,
- Technologies à choisir selon vos compétences.

I.4. Organisation du travail

- Le projet est réalisé **par groupe de 4 étudiants**.
- Chaque groupe est responsable de l'ensemble du projet :
 - ✓ Conception, Développement, Tests, Livraison.

La répartition des tâches au sein du groupe devra être réfléchie et justifiée (ex. : conception, back-end, front-end, coordination...).

I.5. Durée du projet

La durée totale du mini-projet est de **3 jours consécutifs**.

Ce délai court implique :

- Des choix techniques raisonnés,
- Une priorisation des fonctionnalités,
- Une organisation efficace du travail en équipe.

II. Partie 2 : Travail demandé

II.1. Analyse des besoins

Avant toute phase de développement, chaque groupe devra réaliser une **analyse des besoins** afin de bien comprendre ce que doit faire l'application et pour qui elle est conçue.

Cette analyse devra permettre d'identifier clairement :

- Le **public cible** de l'application (utilisateur grand public, passionné de football, utilisateur occasionnel, etc.),
- Les **objectifs principaux** de l'application,
- Les **attentes fonctionnelles** de l'utilisateur,
- Les **contraintes** liées au contexte (temps, données en temps réel, choix technologiques).

Les étudiants sont invités à s'inspirer d'applications ou de sites existants (Google, sites sportifs, applications mobiles de suivi de compétitions) afin d'identifier les fonctionnalités pertinentes, sans pour autant les reproduire à l'identique.

Le résultat de cette analyse devra être formalisé de manière structurée et compréhensible.

II.2. Cahier des charges fonctionnel

À partir de l'analyse des besoins, chaque groupe devra rédiger un **cahier des charges fonctionnel** décrivant précisément les fonctionnalités de l'application.

Sans être limité à cette liste, ce cahier des charges fonctionnel devra notamment inclure les fonctionnalités attendues suivantes :

- La consultation des matchs par phase de compétition :
 - ✓ Phase de groupes,
 - ✓ Seizièmes de finale,
 - ✓ Huitièmes de finale,
 - ✓ Quarts de finale,
 - ✓ Demi-finales,
 - ✓ Petite finale,
 - ✓ Finale ;
- L'affichage des informations d'un match :
 - ✓ Équipes,
 - ✓ Date et heure,
 - ✓ Stade (si disponible),
 - ✓ Score et statut du match (à venir, en cours, terminé) ;
- La mise à jour **en temps réel** des scores et statuts ;
- La navigation entre les différentes phases et matchs ;
- L'ergonomie générale et la lisibilité des informations.

Chaque fonctionnalité devra être décrite de façon claire (objectif, comportement attendu, interactions utilisateur).

II.3. Cahier des charges technique

Chaque groupe devra également produire un **cahier des charges technique** décrivant les choix techniques réalisés pour le projet.

Ce document devra préciser notamment :

II.3.1. Choix technologiques

- Type d'application : web, mobile ou hybride ;
- Technologies utilisées :
 - ✓ Front-end (Frameworks, bibliothèques, langages),
 - ✓ Back-end (si applicable),
 - ✓ Base de données (si applicable) ;
- Mode de récupération des données :
 - ✓ API publique,
 - ✓ API simulée,
 - ✓ Données mockées en temps réel (si nécessaire).

Les choix devront être **justifiés** en tenant compte du temps imparti et du niveau de complexité acceptable.

II.3.2. Architecture générale

- Organisation globale de l'application (architecture client / serveur, architecture MVC, l'application mono-page SPA, API REST, etc.) ;
- Principes de structuration du code ;
- Gestion des échanges de données en temps réel (polling, Web Socket, autre solution adaptée).

II.3.3. Prise en compte de la sécurité

Même dans le cadre d'un mini-projet, la **sécurité applicative** devra être prise en compte dès la phase de conception.

Les étudiants devront identifier et décrire :

- Les **données manipulées** par l'application (scores, informations de matchs, éventuelles données utilisateur),
- Les **risques potentiels** (ex. : exposition d'API, données non contrôlées, attaques basiques côté client).

Le cahier des charges devra préciser au minimum :

- L'utilisation de communications sécurisées (HTTPS) ;
- La protection des clés d'API (non exposées directement dans le code client) ;
- La validation et le contrôle des données reçues ;
- Les bonnes pratiques de base côté front-end (ex. : prévention des injections simples, gestion des erreurs).

L'objectif n'est pas de mettre en place une sécurité avancée, mais de **montrer une prise de conscience et une réflexion cohérente** sur les enjeux de sécurité dans une application réelle.

II.4. Gestion de projet et organisation du travail

Le mini-projet devra être conduit en appliquant des **principes de gestion de projet informatique**, inspirés des méthodes agiles.

L'objectif est de structurer le travail de l'équipe, de favoriser la communication et de garantir l'avancement du projet dans le temps imparti.

II.4.1. Méthodologie de gestion de projet (Agile / Scrum)

Chaque groupe devra expliquer la **méthodologie de gestion de projet** retenue.

Une approche de type **Agile/Scrum** est recommandée et devra être adaptée au contexte court du projet.

Les étudiants devront notamment préciser :

- Les rôles au sein de l'équipe (ex. : responsable de projet / Scrum Master, développeur frontend, développeur back-end, etc.) ;
- L'organisation du travail sur la durée du projet (ex. : mini-sprints, objectifs journaliers) ;
- La manière dont les décisions techniques et fonctionnelles sont prises au sein du groupe ;
- Les moments d'échange et de synchronisation (points rapides, revues, rétrospective en fin de projet).

Il ne s'agit pas d'appliquer Scrum de manière exhaustive, mais d'en **retenir les principes essentiels** et de les adapter intelligemment.

II.4.2. Planification et suivi des tâches (Kanban)

Chaque groupe devra mettre en place un **outil de suivi des tâches** basé sur une approche **Kanban**.

Un outil de type **Trello**, **Jira** ou équivalent pourra être utilisé.

Le tableau Kanban devra au minimum contenir une colonne « *À faire* », une colonne « *En cours* » et une colonne « *Terminé* ».

Les étudiants devront :

- Découper le projet en **tâches claires et concrètes** (fonctionnelles et techniques) ;
- Affecter les tâches aux membres du groupe ;
- Estimer leur charge (même de façon approximative) ;
- Mettre à jour régulièrement l'état d'avancement du tableau.

Le tableau Kanban devra refléter fidèlement l'évolution du projet pendant les 3 jours.

II.4.3. Gestion du code source (GitHub / GitLab)

Le projet devra obligatoirement être versionné à l'aide d'un **outil de gestion de code source** tel que **GitHub** ou **GitLab**.

Les bonnes pratiques suivantes devront être respectées :

- Création d'un **dépôt de projet** dès le début ;
- Utilisation de **commits réguliers**, avec des messages clairs et explicites ;
- Travail collaboratif via :

- ✓ Branches,
- ✓ Ou travail coordonné sur la branche principale selon le niveau du groupe ;
- Résolution des conflits éventuels de manière collaborative.

Chaque groupe devra également :

- Documenter brièvement la structure du dépôt (README),
- Expliquer l'organisation choisie (branches, répartition du travail, règles de commit).

II.4.4. Traçabilité et justification des choix

Les étudiants devront être capables de :

- Justifier leurs choix d'organisation et d'outils ;
- Montrer l'historique du travail réalisé (Kanban, *commits* Git) ;
- Expliquer comment la gestion de projet a contribué (ou non) à la réussite du mini-projet.

Cette partie sera prise en compte dans l'évaluation, au même titre que les aspects fonctionnels et techniques.

II.5. Conception de l'application

Avant et pendant la phase de développement, chaque groupe devra réaliser une **conception complète de l'application**, structurée en **trois volets complémentaires** :

- Conception système,
- Conception de la base de données relationnelle,
- Conception de l'interface utilisateur (front-end).

L'objectif n'est pas de produire une documentation exhaustive, mais une **conception claire, cohérente et exploitable** pour guider le développement.

II.5.1. Conception du système (UML)

Chaque groupe devra réaliser une **conception système** à l'aide de **diagrammes UML**, permettant de comprendre le fonctionnement global de l'application.

Les éléments suivants sont attendus :

a) Diagramme de cas d'utilisation (Use Case)

- Identification des **acteurs** (ex. : utilisateur).
- Description des **cas d'utilisation principaux** :
 - ✓ Consulter les matchs par phase,
 - ✓ Consulter le détail d'un match,
 - ✓ Suivre un match en temps réel, etc.
- Relations entre acteurs et cas d'utilisation.

b) Diagrammes de séquence

- Description des **échanges entre les composants** du système lors de scénarios clés :
 - ✓ Consultation d'une liste de matchs,

- ✓ Mise à jour en temps réel d'un score.
- Mise en évidence des interactions entre :
 - ✓ Interface utilisateur,
 - ✓ Logique applicative,
 - ✓ Source de données / API.

c) Diagrammes d'activité

- Représentation du **flux des actions** pour certains parcours utilisateur (ex. : navigation entre phases, affichage d'un match en cours).
- Identification des décisions et des enchaînements logiques.

d) Diagramme de classes

- Identification des classes principales de l'application ;
- Définition des attributs et des relations entre classes ;
- Cohérence entre les classes et les données manipulées.

e) Diagramme des entités

- Identification des entités métier (match, équipe, phase, score, etc.) ;
- Relations entre entités ;
- Alignement avec la future conception de la base de données.

Les diagrammes devront être lisibles, cohérents entre eux et adaptés au périmètre du projet.

II.5.2. Conception de la base de données relationnelle (Méthode MERISE)

Chaque groupe devra réaliser une **conception de la base de données** en s'appuyant sur la méthode **MERISE**.

a) Modèle Conceptuel de Données (MCD)

- Identification des entités ;
- Définition des associations ;
- Cardinalités ;
- Attributs principaux.

b) Modèle Logique de Données (MLD)

- Transformation du MCD en tables relationnelles ;
- Définition des clés primaires et étrangères ;
- Respect des règles de normalisation.

c) Modèle Physique de Données (MPD)

- Traduction du MLD dans un modèle physique adapté au SGBD choisi ;
- Types de données ;
- Contraintes (NOT NULL, clés, index éventuels).

d) Dictionnaire de données

- Description de chaque donnée :

- ✓ Nom, type, taille, min, max, description, contraintes éventuelles, format.
- Cohérence avec les modèles précédents.

L'ensemble devra être clair, structuré et directement exploitable pour l'implémentation.

II.5.3. Conception de l'interface utilisateur (Front-end)

Chaque groupe devra réaliser une **conception du front-end** afin de définir l'expérience utilisateur et l'aspect visuel de l'application.

a) Zonings

- Découpage des écrans principaux en zones fonctionnelles ;
- Identification des éléments clés (navigation, contenu, informations de match, etc.).

b) Wireframes

- Représentation simplifiée des écrans ;
- Organisation des informations ;
- Parcours utilisateur.

c) Charte graphique

- Choix des couleurs ;
- Typographies ;
- Icônes ;
- Cohérence visuelle globale.

d) Maquettes graphiques

- Maquettes plus détaillées des écrans principaux ;
- Respect de la charte graphique ;
- Lisibilité et ergonomie.

Les outils de conception sont laissés au choix des groupes (ex. : Figma, Adobe XD, papier/crayon numérisé, etc.).

II.5.4. Cohérence globale de la conception

L'ensemble des conceptions devra être :

- **Cohérent entre les différents volets** (UML ↔ BDD ↔ Front-end) ;
- Adapté aux fonctionnalités réellement développées ;
- Compréhensible par un tiers.

La qualité de la conception sera évaluée autant que le résultat fonctionnel final.

II.6. Développement de l'application

Chaque groupe devra réaliser le **développement complet de l'application**, en s'appuyant sur les phases de conception précédemment définies.

Le développement devra couvrir :

- Le **front-end** (interface utilisateur),
- Le **back-end** (logique applicative, accès aux données),
- La **base de données** (si utilisée).

L'application développée devra être fonctionnelle, stable et cohérente avec les documents de conception.

II.6.1. Développement Front-end

Le développement front-end devra permettre à l'utilisateur de :

- Naviguer facilement entre les différentes phases de la compétition ;
- Consulter les listes de matchs ;
- Visualiser les informations détaillées d'un match ;
- Voir les scores et statuts se mettre à jour en **temps réel**.

a) Exigences techniques

Les étudiants devront :

- Respecter la structure définie lors de la conception ;
- Organiser le code de manière claire et maintenable ;
- Séparer la logique métier de l'affichage ;
- Gérer correctement les états de l'application (chargement, erreurs, données indisponibles).

b) Sécurité côté Front-end

Le front-end devra intégrer les **bonnes pratiques de sécurité** suivantes :

- Validation des données affichées et reçues ;
- Prévention des failles courantes côté client (ex. : XSS basiques) ;
- Gestion propre des erreurs sans divulgation d'informations sensibles ;
- Absence de clés d'API ou de secrets directement exposés dans le code client.

II.6.2. Développement Back-end

Le développement back-end devra assurer :

- La récupération et/ou la gestion des données de matchs ;
- La fourniture des données au front-end via une API ;
- La mise à jour des données en temps réel (ou quasi temps réel).

a) 22.1 API et logique applicative

Le back-end devra :

- Exposer des *endpoints* clairs et cohérents ;
- Respecter une structure logique (routes, services, contrôleurs, etc.) ;
- Gérer les erreurs et les cas particuliers ;
- Contrôler les données échangées avec le front-end.

b) Sécurité côté Back-end

Une attention particulière devra être portée à :

- La validation et le filtrage des données entrantes ;
- La protection des endpoints (limitation basique, contrôle d'accès si nécessaire) ;
- La gestion sécurisée des clés, tokens et paramètres sensibles ;
- L'utilisation de communications sécurisées (HTTPS).

II.6.3. Gestion et sécurisation de la base de données

Si une base de données est utilisée, celle-ci devra être :

- Conforme au modèle de données conçu ;
- Correctement structurée ;
- Sécurisée.

Les bonnes pratiques suivantes sont attendues :

- Utilisation de requêtes sécurisées (prévention des injections SQL) ;
- Définition de contraintes d'intégrité (clés, relations) ;
- Limitation des droits d'accès à la base de données ;
- Protection des données stockées (pas de données inutiles ou sensibles).

II.6.4. Sécurité globale de l'application

La sécurité devra être prise en compte de manière **transversale** tout au long du développement.

Chaque groupe devra démontrer :

- Une compréhension des **risques de sécurité basiques** ;
- Une cohérence entre les choix techniques et les mesures de protection mises en place ;
- Une application fonctionnelle sans failles évidentes.

L'objectif n'est pas de mettre en œuvre une sécurité avancée, mais d'appliquer des **principes fondamentaux de sécurité applicative** adaptés à un mini-projet.

II.6.5. Tests et vérifications

Avant la livraison finale, les groupes devront :

- Tester les fonctionnalités principales ;
- Vérifier la stabilité de l'application ;
- S'assurer que les données sont correctement protégées et manipulées.

Les tests pourront être manuels et ciblés, en lien avec les fonctionnalités développées.

Livrables attendus :

Chaque groupe devra fournir les **livrables suivants**, organisés de manière claire et structurée.

➤ Livrables de conception :

- Analyse des besoins
- Cahier des charges fonctionnel
- Cahier des charges technique
- Documents de conception :
 - UML : diagrammes de cas d'utilisation, de séquence, d'activité, de classes, des entités ;
 - MERISE : MCD, MLD, MPD, dictionnaire de données ;
 - Conception front-end : zonings, wireframes, charte graphique, maquettes.

➤ Livrables de gestion de projet

- Description de la méthodologie de gestion de projet utilisée (ex. Agile / Scrum)
 - Tableau Kanban (Trello, Jira ou équivalent) montrant : les tâches, leur état d'avancement, et la répartition du travail
 - Dépôt GitHub / GitLab avec : historique des commits et organisation du projet

➤ Livrables de développement

- Code source complet de l'application (front-end et back-end)
- Application fonctionnelle (locale ou déployée si possible)
- Respect des principes de sécurité abordés dans le sujet

Modalités de rendu :

➤ Rendu des livrables

Les livrables devront être remis selon les modalités précisées par le formateur :

- lien vers le dépôt GitHub / GitLab,
- documents de conception (PDF ou équivalent),
- lien vers l'application ou démonstration locale.

➤ Présentation orale individuelle

Chaque élève devra réaliser une **présentation orale individuelle** d'une durée de **30 minutes**.

Ensuite, le jury effectuera un **entretien technique** (sous forme d'échanges et questions) d'une durée de **30 minutes**.

- Les élèves **peuvent utiliser le même support PowerPoint**, réalisé par le groupe.
- La présentation doit toutefois refléter la **compréhension personnelle** de l'élève.
 - Chaque élève devra être capable d'expliquer les choix fonctionnels, les choix techniques, la conception, la gestion de projet, la sécurité, sa contribution personnelle au projet.

La présentation pourra s'appuyer sur une démonstration de l'application, des diagrammes de conception et de l'architecture, des captures du Kanban, des extraits du dépôt Git.

Critères d'évaluation :

L'évaluation portera à la fois sur :

- La **qualité du travail de groupe**,
- La **compréhension individuelle** du projet,
- La **posture professionnelle** adoptée.

Les critères principaux sont :

- Compréhension du besoin et pertinence de l'analyse ;
- Qualité et cohérence des conceptions (UML, MERISE, front-end) ;
- Qualité du développement (fonctionnalités, structure, lisibilité) ;
- Prise en compte de la sécurité ;
- Organisation et gestion de projet ;
- Qualité de la présentation orale individuelle.



Ryadh HADJ MOKHNECHE